

COMP1549: Advanced Programming

Part a: Introduction to Design Patterns

Part b: GRASP

Part c: Factory Pattern

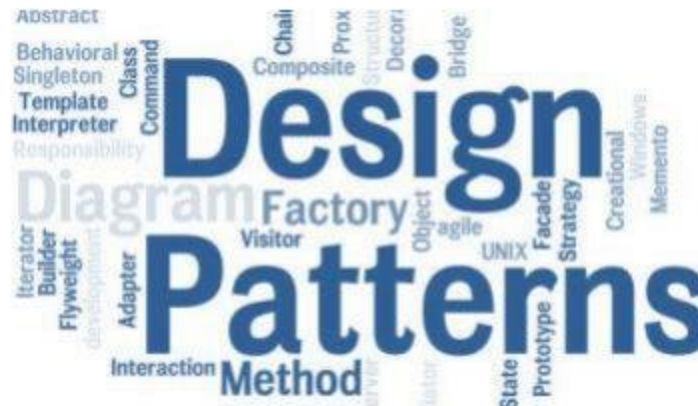
Part d: Singleton Pattern

Part e: Other Patterns

Dr Markus Wolf

5th March, 2026 - Week 8

Introduction to Design Patterns



Why Design Patterns?

- Imagine that I give everyone here a program to implement
- I even specify the programming language
 - Let's say it's Java
- Will I end up with X identical implementations?

Why Design Patterns?

- There are many ways in which to solve a problem
 - Especially in software development
- Knowing what features a language supports and how to apply them will result in better code
- Another important aspect is how elegantly a solution is implemented
 - Flexible
 - Reusable
 - Maintainable
 - Reliable

What are Design Patterns?

- Design patterns are named problem/solution pairs that codify good advice and principles for assigning responsibilities (mainly to classes)
- Encapsulate recurring problems and their solutions
- Build on past experience
 - Good solutions that have proven themselves
- Balanced assignment of responsibilities is a very important aspect of a design (and thus implementation)

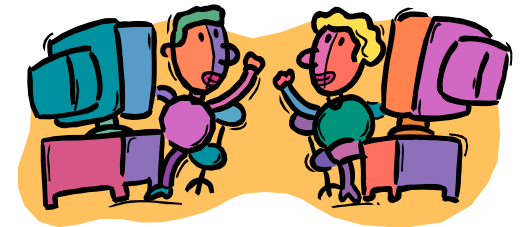
What are Design Patterns?

- Where is the design patterns library, because I can't wait to get hold of this code?
 - Not a reusable code solution 
 - Not a finished design that can be directly translated into code 
 - A description or template for solving a problem
 - ❖ Often described with the aid of UML diagrams

Reuse at design,
not code level

Constituents of a Design Pattern

- Name (we said it is a named problem/solution pair)
 - Why named?
 - ❖ Helps understanding and remembering
 - ❖ Facilitates communication
- Problem
 - What are we trying to solve?
- Solution
 - How the problem is solved



How Did We Get Design Patterns?

- Notion of patterns originated with architectural patterns of Christopher Alexander
 - “Each pattern describes a problem which occurs over and over again, and then describes the core of the solution”
- First introduced to the realm of software development in the 1980s by Kent Beck (the man who gave us extreme programming and unit testing)

How Did We Get Design Patterns

- Major milestone was the publication of a book by Gang of Four (GoF, Gamma, Helm, Johnson & Vlissides)
 - *Design Patterns: Elements of Reusable Object-Oriented Software*
 - ❖ Published in 1994
 - ❖ Considered the “Bible” of design pattern books
 - ❖ Outlines 23 patterns
- Many authors have described design patterns since
 - Some more influential ones include:
 - ❖ Craig Larman – GRASP
 - ❖ Martin Fowler – Enterprise Patterns

Anti-Patterns

- There are also anti-patterns
 - A pattern that appears obvious, but is ineffective or far from optimal in practice
 - May appear to be beneficial at first, but ultimately brings more negative consequences than positive

Responsibilities

- ◉ Designing a software solution requires assignment of responsibilities
 - Applies at all levels (e.g. methods, classes, components, subsystems)
- ◉ Responsibility of
 - *Doing*
 - ❖ Performing some calculation
 - ❖ Coordinating some activity
 - *Knowing*
 - ❖ Keeping encapsulated data
 - ❖ Know related objects
- ◉ Responsibility-Driven Design regards OO designs as *communities of collaborating responsible objects*

Future-Proofing

- ◉ How flexible and future-proof should our designs be?
- ◉ According to Craig Larman:
 - Novice developers tend towards brittle designs
 - Intermediate developers tend toward overly fancy and flexible, generalised ones
 - ❖ In ways that never get used
 - Expert designers choose with insight
 - ❖ Perhaps a simple and brittle design whose cost of change is balanced against its likelihood

GRASP



GRASP

- A set of principles (or design patterns) for assigning responsibility
- General Responsibility Assignment Software Patterns (Principles)
- Nine patterns presented to us by Craig Larman
- Too general to be captured as UML diagrams
- Some you will have come across before (maybe not tagged as a pattern)
- Let's grasp GRASP

Low Coupling

- Coupling refers to how strongly an element (class, component or subsystem) is connected to, has knowledge of, or relies on other elements
- Problem:
 - When elements are highly coupled, a change in one may enforce changes in others
 - Highly coupled elements are not very reusable – if I want to reuse one, I have to get all its related elements

Low Coupling

- Solution:

- Assign responsibilities such that coupling is low
- Leads to low impact change and high reusability, as elements are more independent

High Cohesion

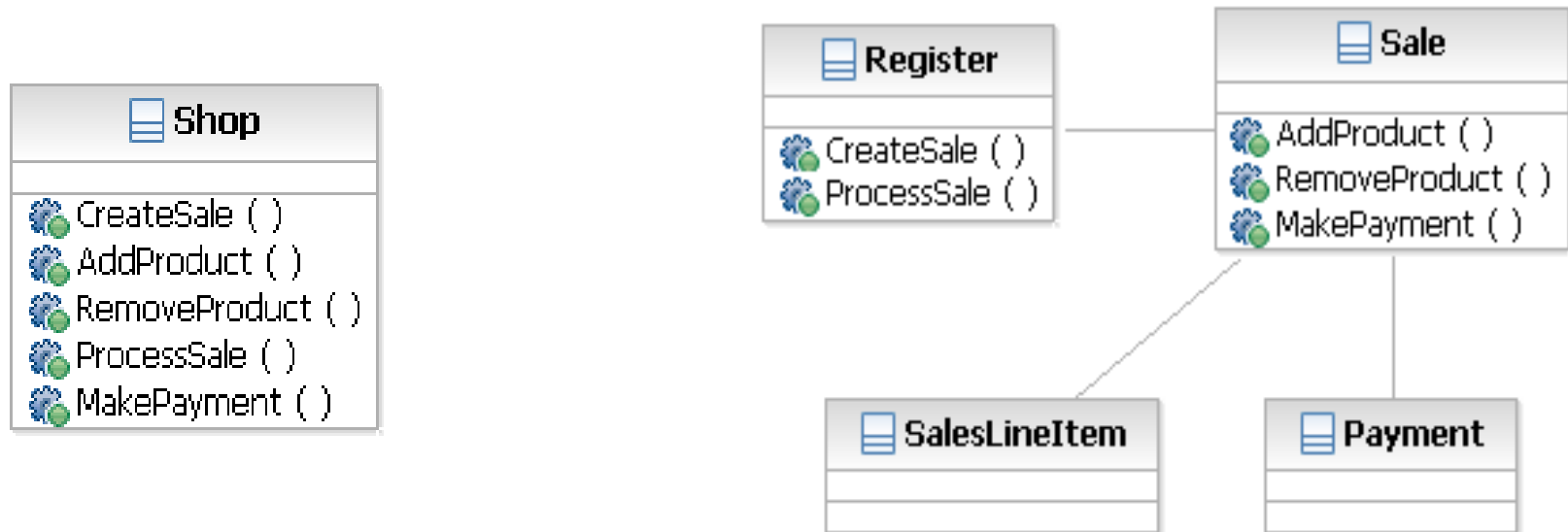
- Cohesion or functional cohesion refers to how focussed and related an element's responsibilities are
- Problem:
 - An element has too much responsibility (work)
 - ❖ What happens to you when you do too much or very unrelated work?
 - An element is responsible for a variety of unrelated tasks – lacks focus
 - Elements with low cohesion are hard to reuse and maintain

High Cohesion

- Solution:
 - Assign responsibilities such that cohesion remains high
 - This makes an element more reusable, maintainable and easier to understand
- A highly cohesive class should have a relatively small number of related methods and delegate work to others where a task requires a considerable amount of work

High Cohesion Example

- Where do we have higher cohesion?



Creator

● Problem:

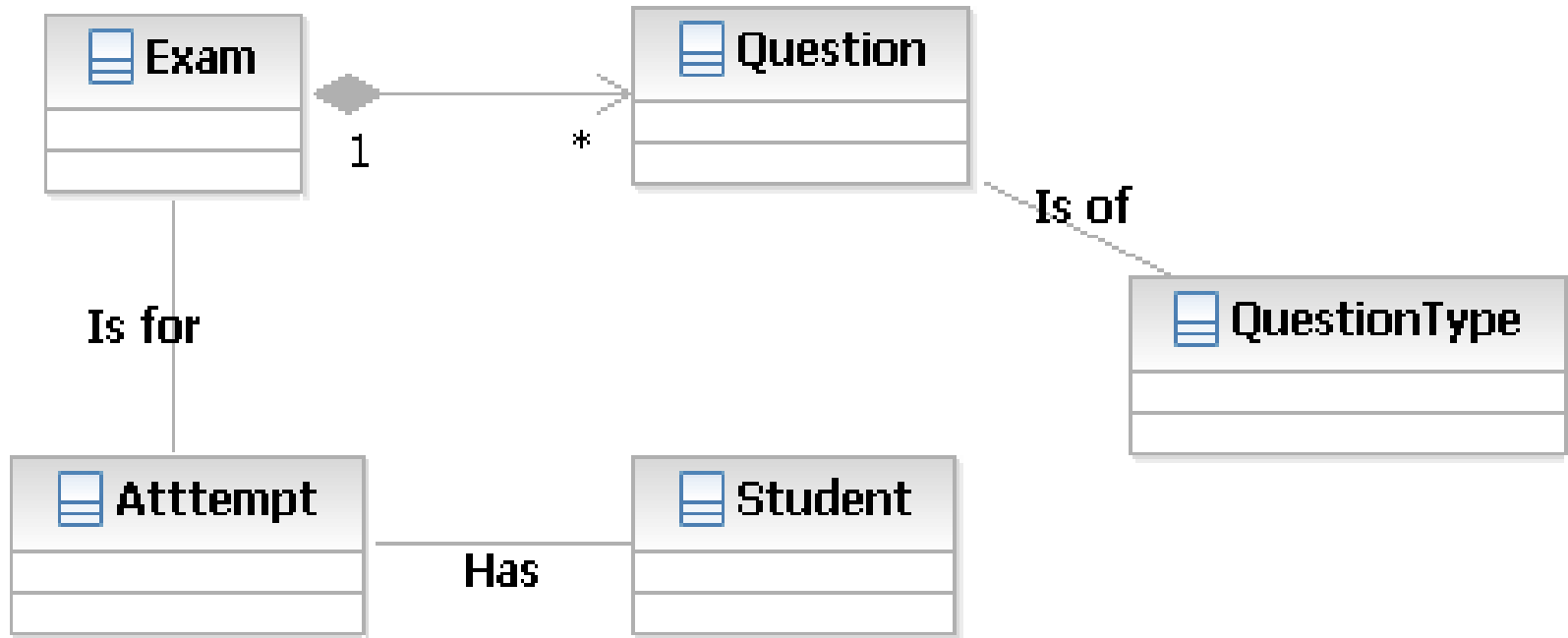
- Who should be responsible for creating instances of a class

● Solution:

- Class B should be responsible for instantiating class A if:
 - ❖ B is composed of A
 - ❖ B records A
 - ❖ B closely uses A
 - ❖ B has the required initialising data for A

Creator Example

- Who should create new instances of *Question*?



Information Expert

- Problem:

- How should responsibility be assigned to elements

- Solution:

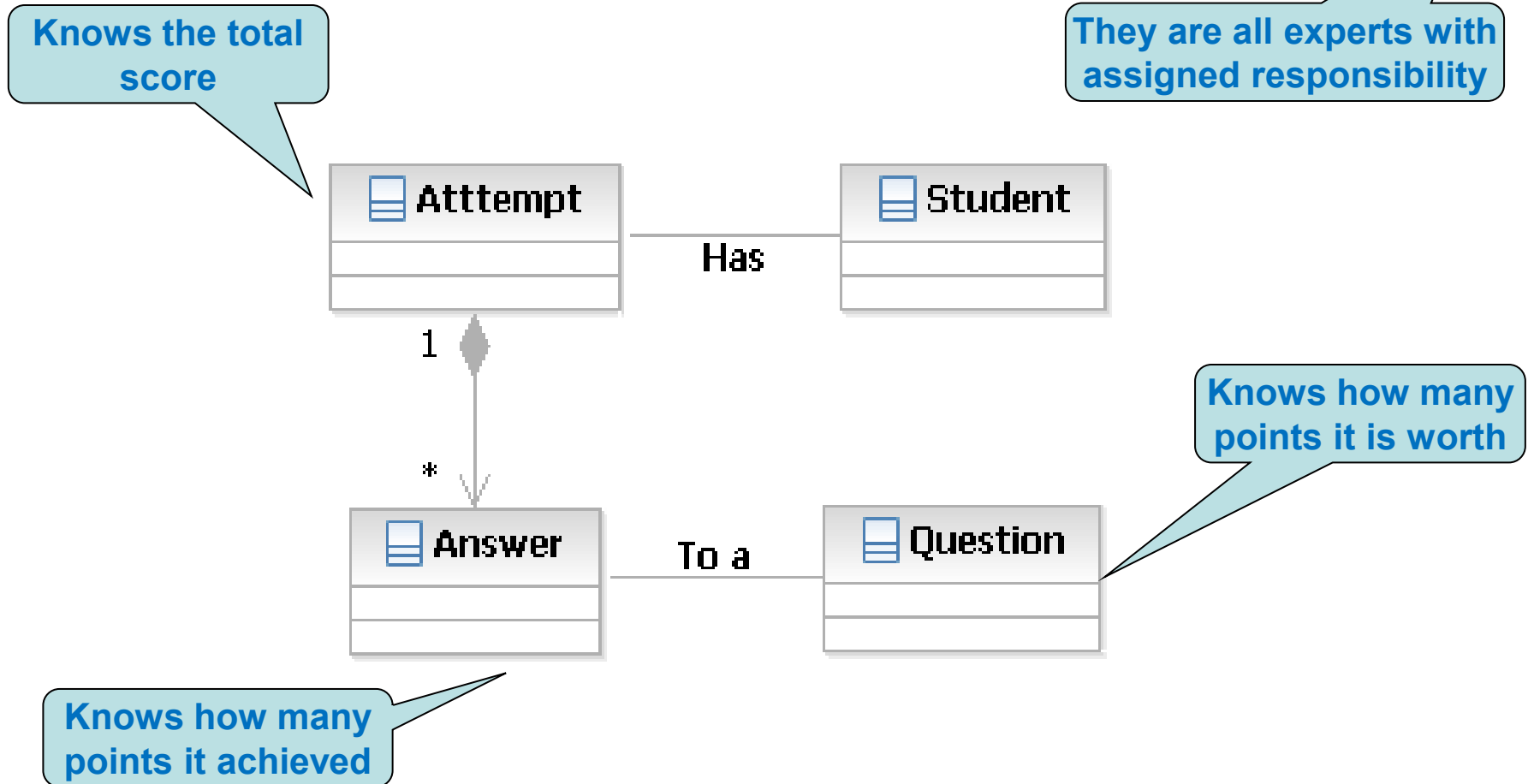
- Responsibility should be assigned to the information expert – the element that has the information required to fulfil the responsibility

Information Expert

- In object-oriented software all elements are “alive” - can do things that the real-life objects they represent have done to
 - Can do things
 - Can have responsibility
 - E.g. an exam can calculate its own total
 - Known as the “Do It Myself” strategy

Information Expert Example

- Who should know the total score?



Controller

● Problem:

- Who should be responsible for processing system events

● Solution:

- Introduce a class behind the UI layer that dispatches events accordingly
- Could represent:
 - ❖ Overall system or major subsystem (Façade Controller)
 - ❖ A use case scenario (Use-Case Controller)
 - ❖ A session (Session Controller)

Polymorphism in Programming

- Poly (many) morphism (shapes)
- Concept of polymorphism is supported by object-oriented programming languages (e.g. Java, C#)
 - Possible to use objects of different types
 - ❖ If one is a subtypes (inheritance)
 - ❖ If they implement the same type (interface or abstract class)

Polymorphism

● Problem:

- Handle alternatives based on type in a flexible way
- Create pluggable components

● Solution:

- Assign type-specific responsibility to a type and use polymorphism to implement the solution
- Works when alternative behaviour varies by type
 - ❖ E.g. calculating the area of a Triangle and a Rectangle class

Polymorphism

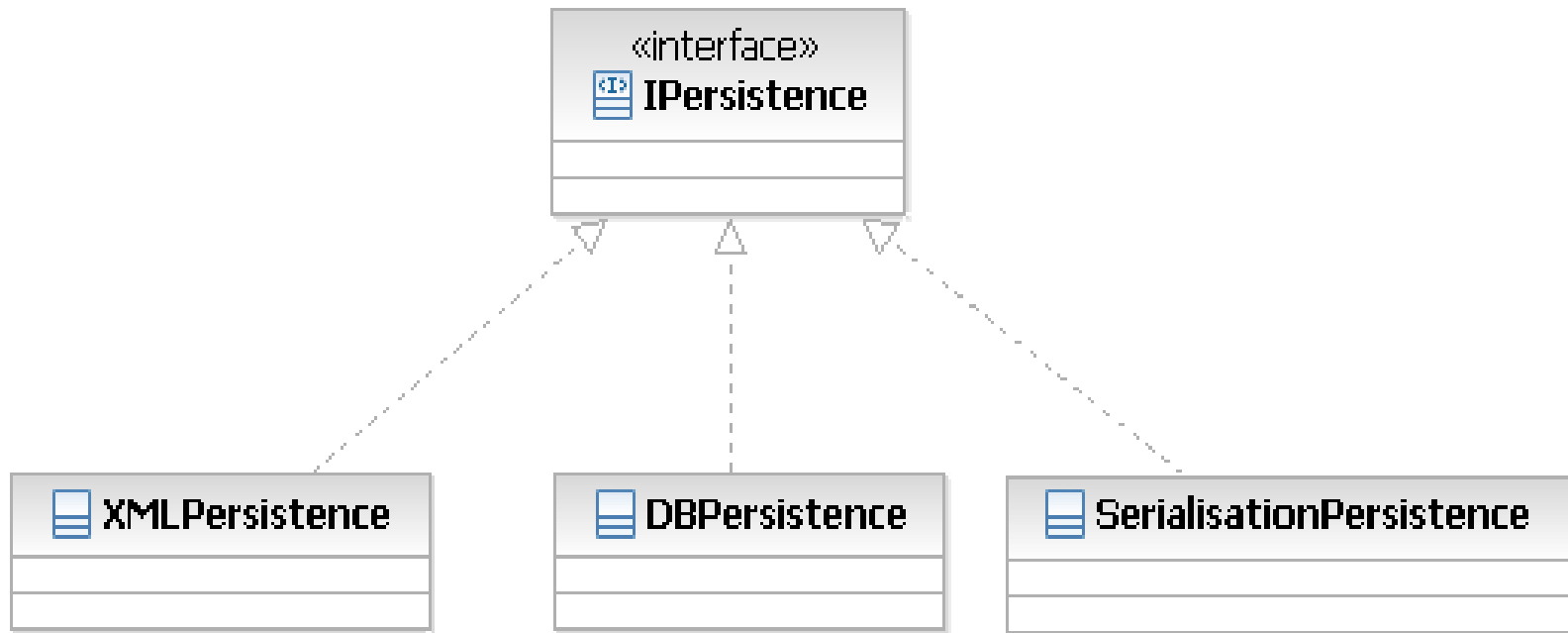
- ◉ Instead of having conditional logic to check for type and take appropriate action, program to an interface and place the differing behaviour in the type
- ◉ All alternative types need to provide same type-specific operation

Polymorphism

- Polymorphism is generally implemented using interfaces or abstract classes
 - Using interfaces overcomes the single inheritance of Java
 - Keeps it flexible for future evolution
- Using polymorphism makes it easy to extend an application
 - New implementations can be introduced without affecting existing clients

Polymorphism Example

- Through polymorphism, we have different persistence classes with similar, but varying, behaviour



Pure Fabrication

● Problem:

- The Information Expert offers a solution for assigning responsibility to a domain class, but this violates High Cohesion, Low Coupling and/or other design goals (e.g. reuse)

● Solution:

- Assign the responsibility to an artificial or convenience class
 - ❖ It does not represent a domain concept
 - ❖ You make it up for your convenience – it is a fabrication

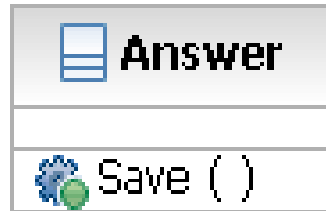
Pure Fabrication

- Pure Fabrication classes are normally related to functionality and behaviour
 - Doesn't represent a thing in our domain
- Allows the developer to group related behaviour
- Should still be cohesive
 - The behaviour placed in a fabrication should be related or reassigned to more fabricated classes

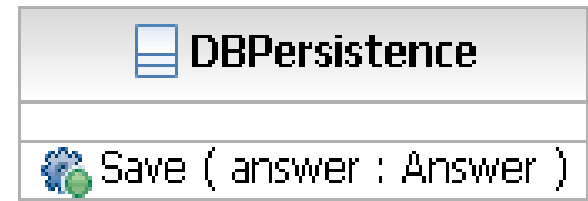
Pure Fabrication Example

- According to the Information Expert, if we want to persist an answer, the class that has the data which should be saved is *Answer* itself

Would this be a good idea?



Would create high coupling to persistence-related classes (e.g. Connection classes, Command classes)



A pure fabrication – but it makes the **Answer** more reusable and our **Persistence** class is still cohesive

Indirection

- Problem:
 - Two or more elements have high coupling and low reuse potential
- Solution:
 - Create an intermediate element to mediate between the other elements, which are no longer directly coupled
- Is the basis for many design patterns
 - E.g. Facade, Observer

Indirection

- “All problems in computer science can be solved by another level or ***indirection***”

Butler W. Lampson (Computer Scientist)

- “... except for the problem of too many layers of ***indirection***”

Kevlin Henney (Author)

- “Most problems in performance can be solved by removing another layer of ***indirection***”

David Wheeler (Computer Scientist)

Protected Variation

- Problem:
 - Making changes to an element has an undesirable impact on other elements
- Solution:
 - Identify points of predicted variation or instability and assign responsibility to create a stable interface around them
- Also known as *information hiding*

Protected Variation

- Provides flexibility and protection from variation in data, behaviour, software components, operating systems, hardware
- Very general principle
 - Applied in data encapsulation, polymorphism, interfaces, configuration files, and more
- Special form of Protected Variation is Law of Demeter (Don't Talk to Strangers)
 - Protect from structure changes by avoiding designs that traverse long object structures or send messages to indirect objects (strangers)
 - ❖ E.g.
`foo.getA().getB().getC().getD().doStuff();`

Protected Variation Example

- An example of Protected Variation is the externalisation of the variant
 - The system/application is parameterised at runtime with data read in from an outside source (e.g. configuration file)
- The language for an application that supports a number of languages can be read in from a configuration file
 - The application could be set to use an English, Spanish or German GUI, without having to change code

Factory Pattern



Remember the
Creator GRASP
pattern?

Factory Pattern

● Problem:

- Who should be responsible for creating objects when there are special considerations
 - ❖ E.g.
 - Complex creation logic
 - Separate creation responsibilities for better cohesion
 - Polymorphic creation (use alternative implementations)

● Solution:

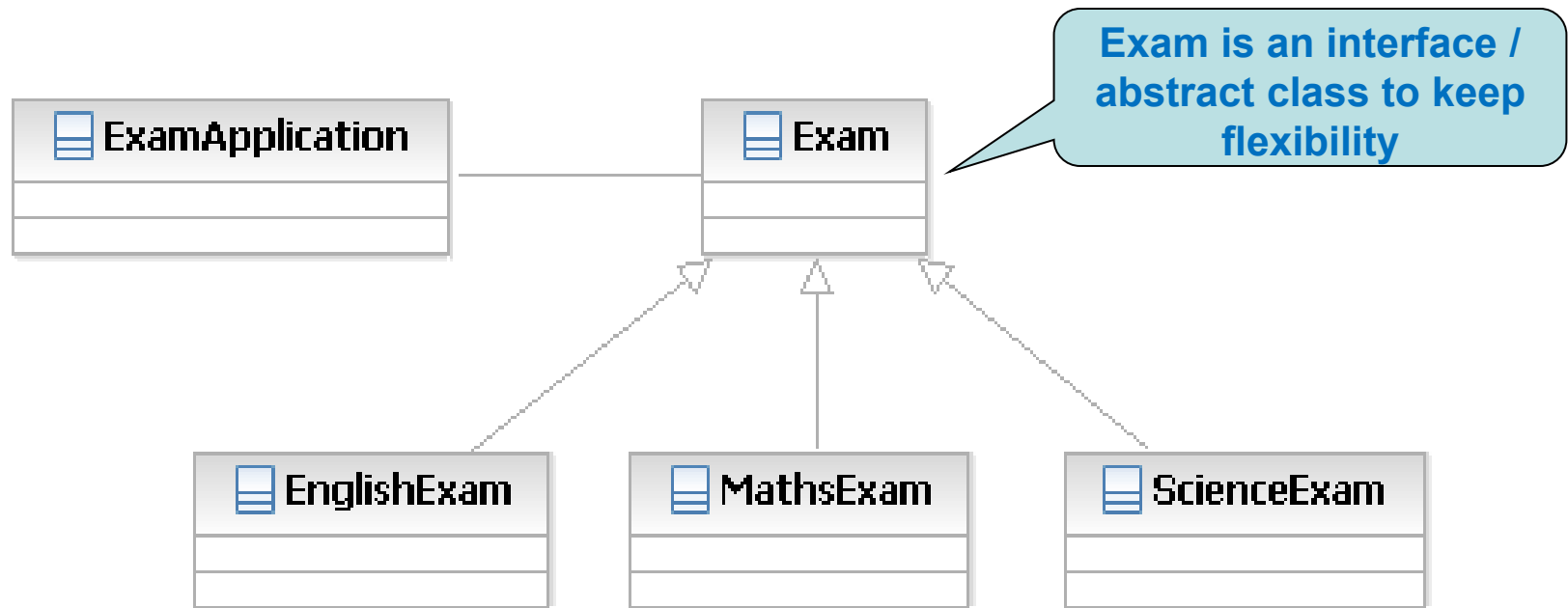
- Create a pure fabrication object (factory) that handles creation

Factory Pattern

- Also called Simple Factory or Concrete Factory
- A simplification of the GoF Abstract Factory pattern
- Its main aim is to decouple code that makes use of an implementation from the actual implementation
- Can also be used to improve performance
 - Caching and reusing objects

Example

- Imagine an application for sitting exams
- Different exams exist for different subjects
 - With a different class per exam



Example

- On the previous slide it looked as if the ExamApplication and the different implementations are decoupled, but...
- You could have code, as the following, in your ExamApplication:

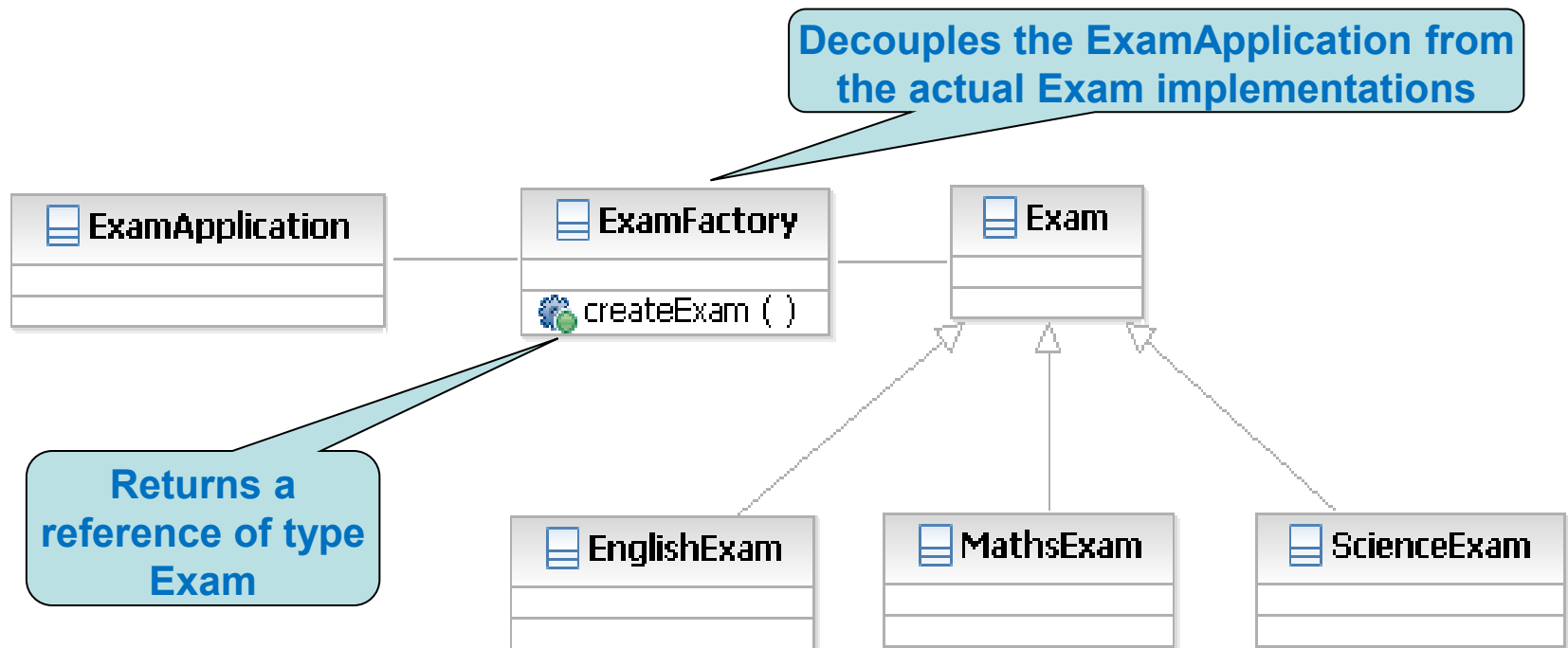
```
Exam exam;  
  
if(examType == ExamType.English) {  
    exam = new EnglishExam();  
}  
  
else if(examType == ExamType.Maths) {  
    exam = new MathsExam();  
}
```

ExamType is an
enumeration

Not very flexible
– what if new
exams are
introduced?

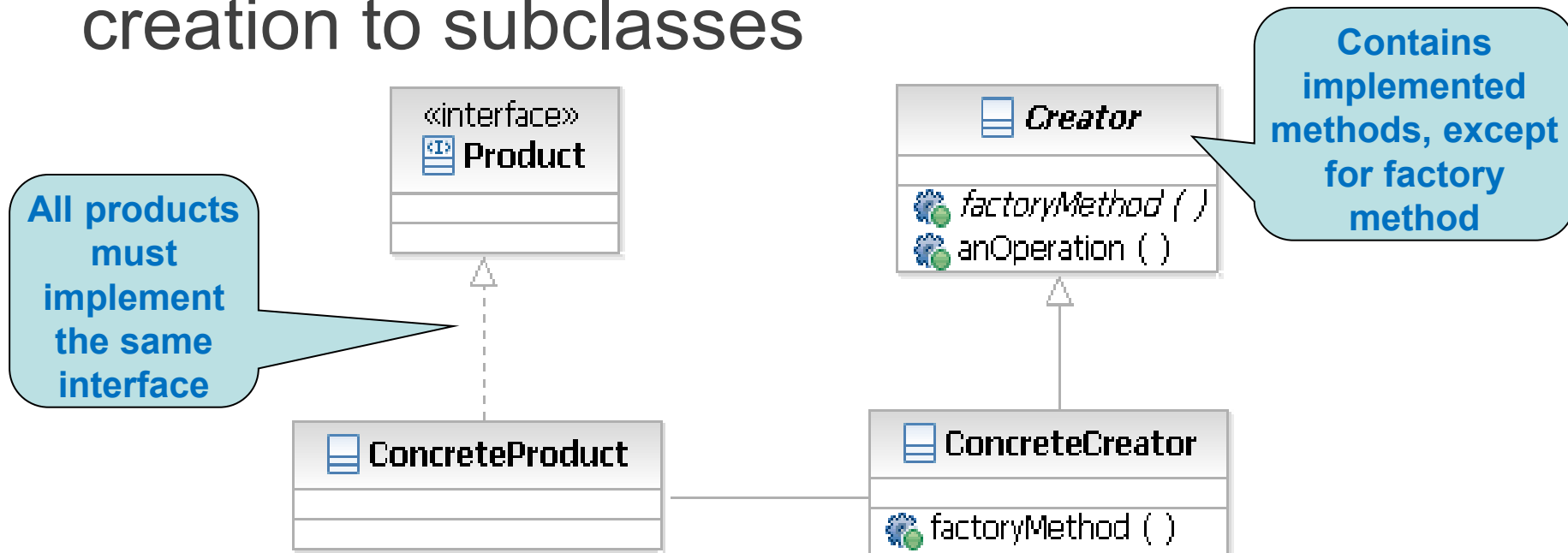
Example

- If we create a factory, this code is placed in a class whose responsibility is the creation of Exams



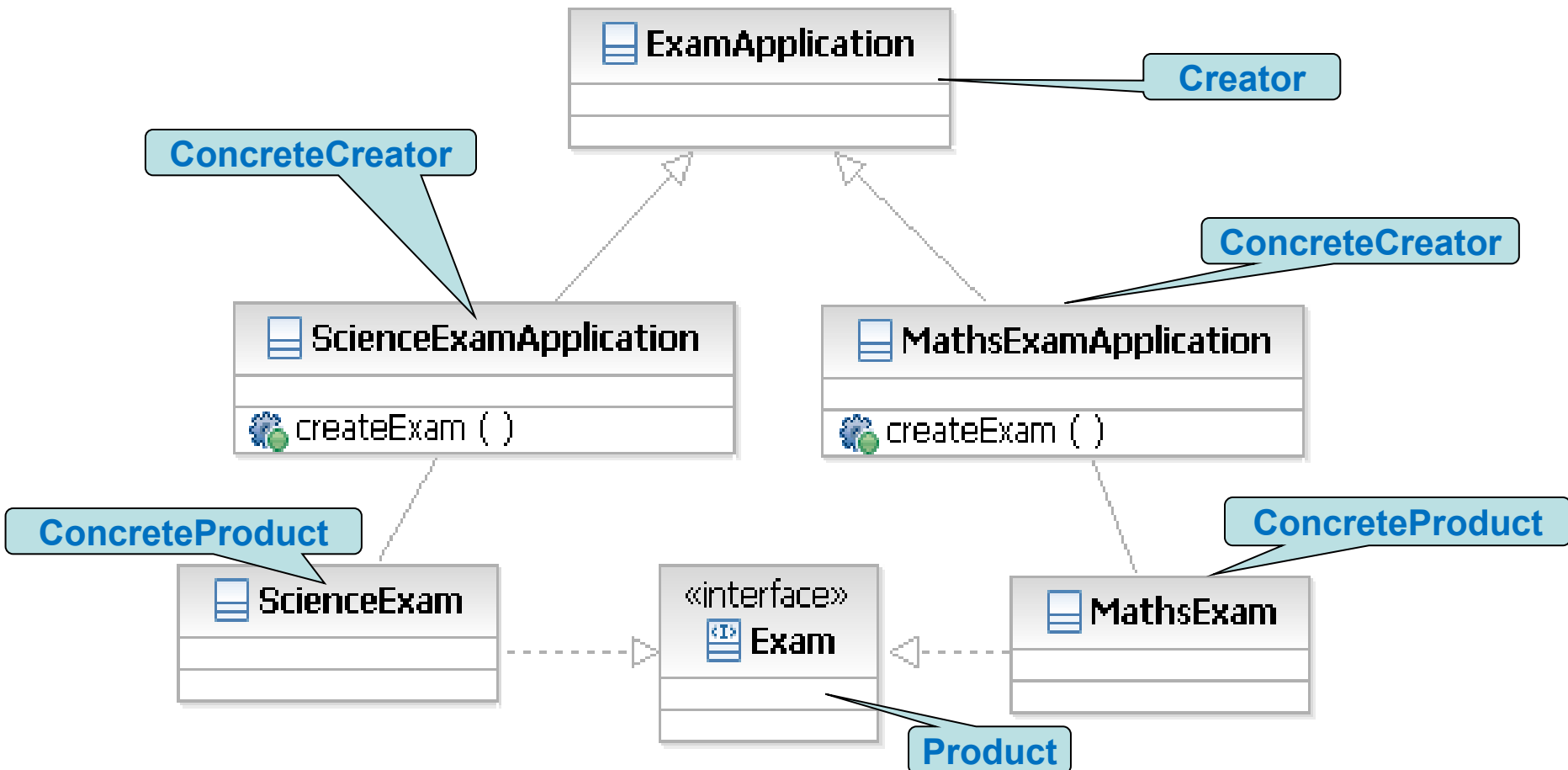
Factory Method Pattern

- A GoF design pattern
- The problem is the same, but the implementation is different
- Encapsulates object creation by delegating creation to subclasses



Factory Method Example

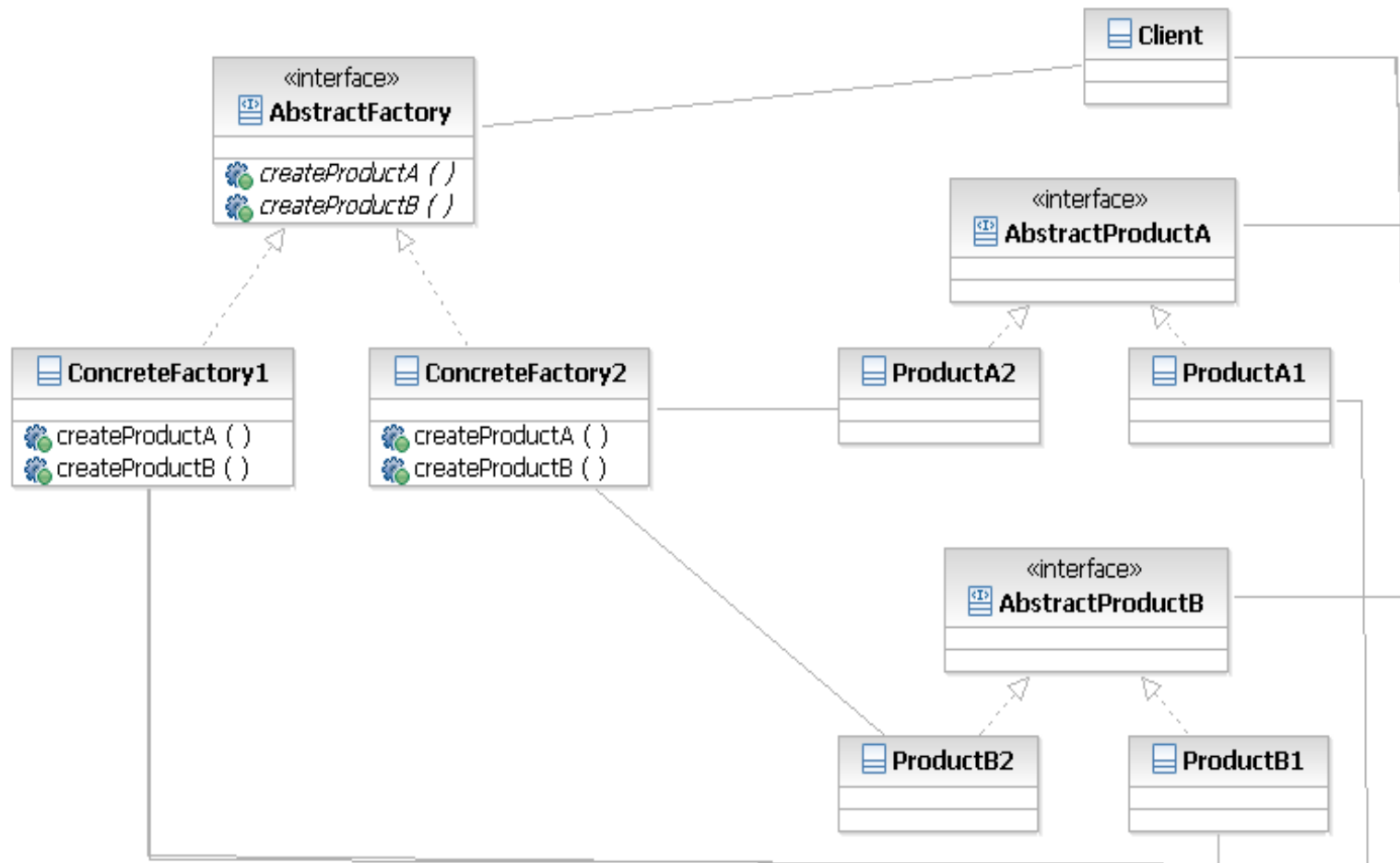
- What would our case-study look like?



Abstract Factory Pattern

- Yet another factory pattern
- A GoF design pattern
- Provides an interface for creating families of related or dependent objects without specifying their concrete classes
- The difference with this and the previous factory patterns is that we deal with creating a set of objects
 - Has a factory for each set of related objects and an abstract factory above them

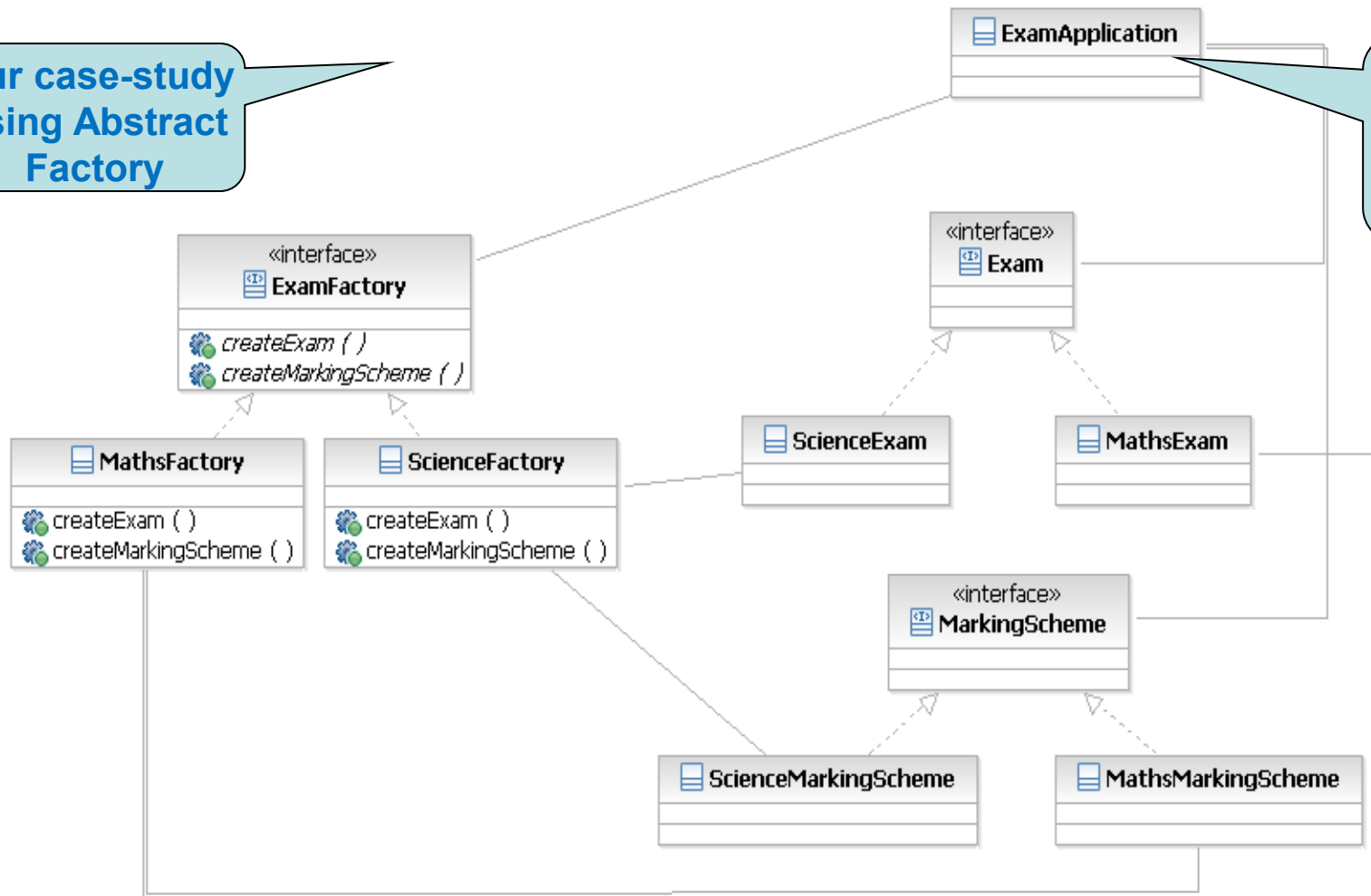
Abstract Factory Pattern



Abstract Factory Example

Our case-study
using Abstract
Factory

The client is
only coupled to
interfaces



Singleton Pattern



Singleton Pattern

- A Creational Pattern

- Problem

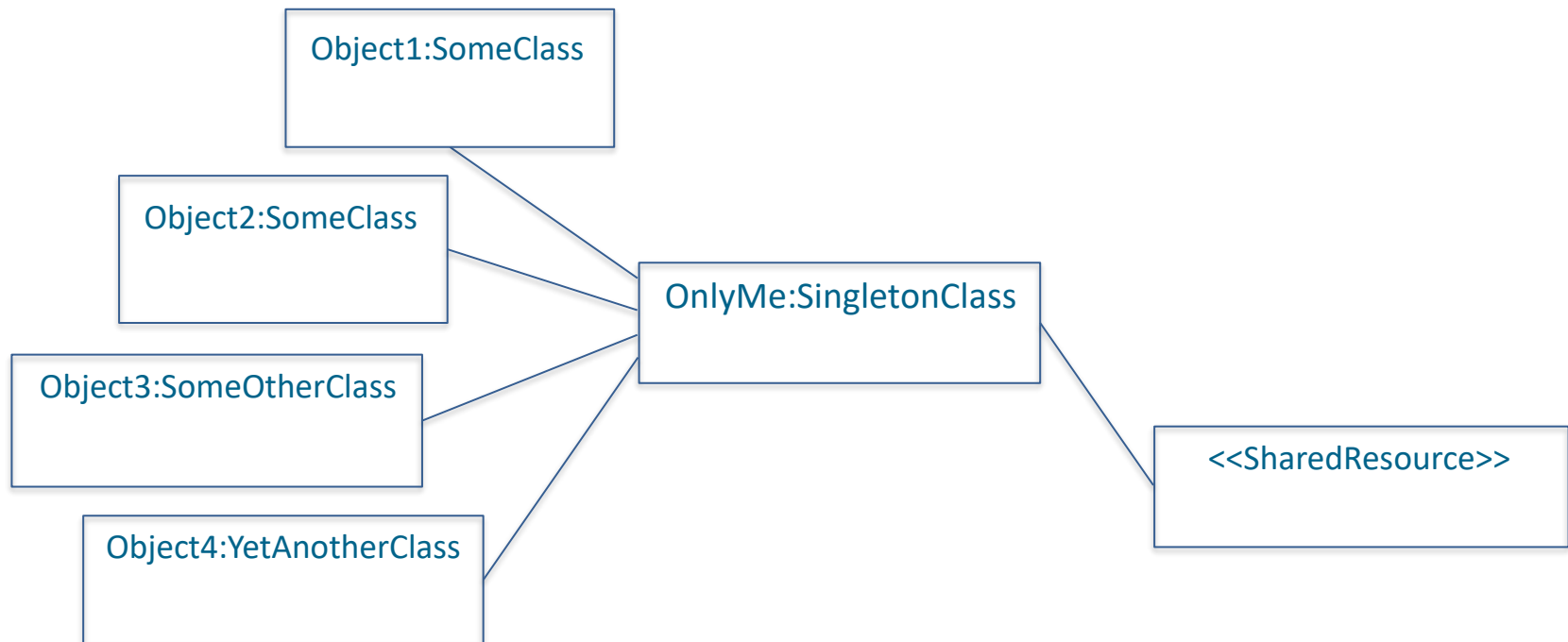
- Need to ensure that **only one instance** of a particular class ever exists in a program
- This might be because there is a need to control access to some resource such as a database, communications line, print queue
- We'd like to make it illegal to have more than one, just for safety's sake!
 - ❖ We could have more, but control the number of instances that exist
- Access to this single instance is required from lots of places in your program

Singleton Pattern

- Why is this a problem?
 - It is difficult to deal with different objects floating around if they are essentially the same - inconsistencies
 - Creating lots of objects affects performance
 - Extra objects take up memory

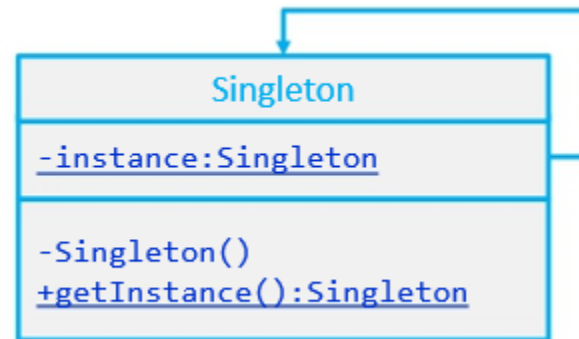
The need for a Singleton

- We want to ensure that only one instance of SingletonClass can ever exist AND it is easy for the other objects to get access to that instance



Singleton - Solution

UML for the SingletonClass

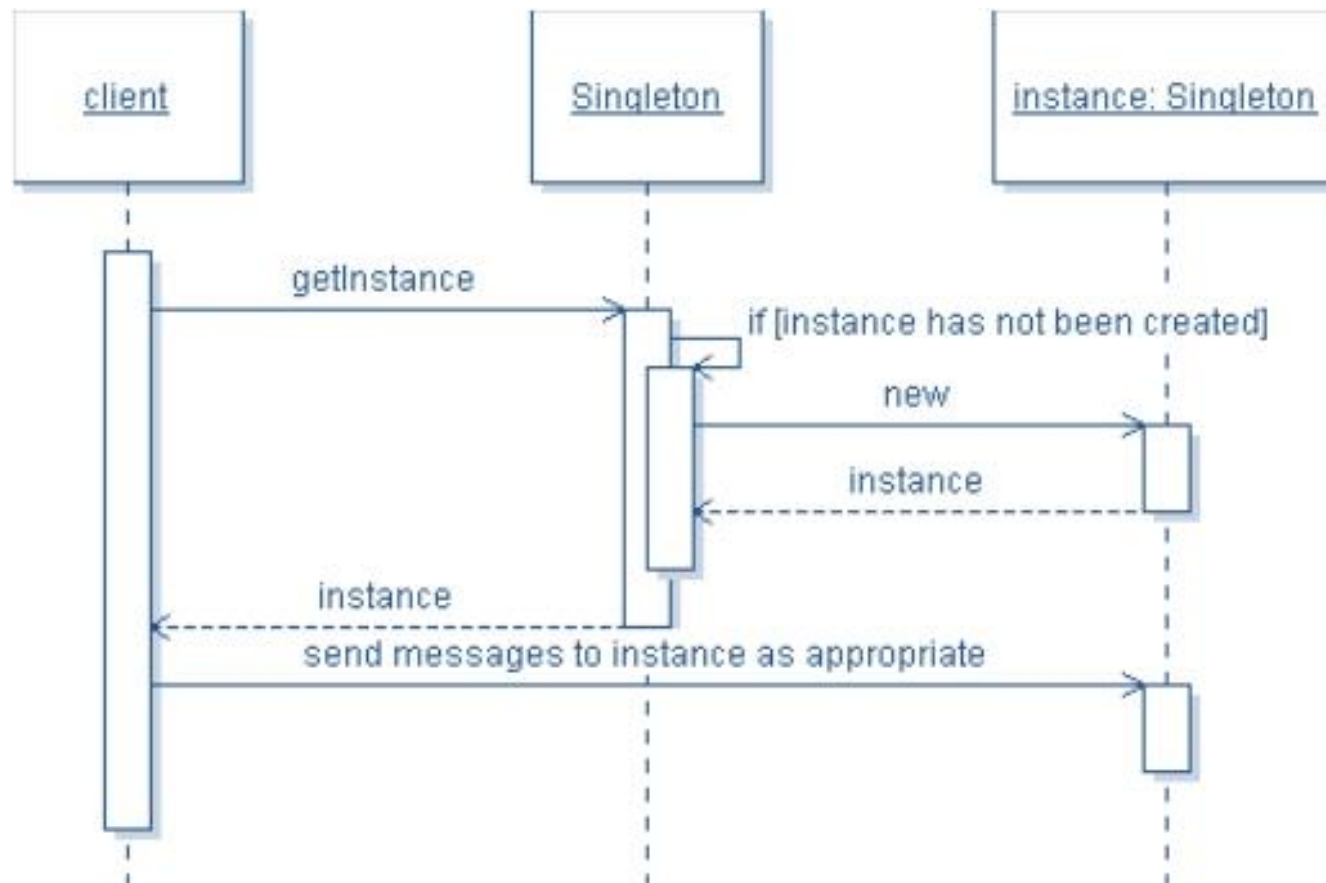


Note: that underlining indicates static members

Create a class with:

- a **private constructor** so other objects can't create an instance of it, and
- provide a **static method** that creates an instance of the class if it doesn't already exist and returns a reference to the single instance

Singleton – Sequence Diagram



Singleton – Outline Code

```
public class Singleton {  
  
    private Singleton instance = null;  
  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (instance == null)  
            instance = new Singleton();  
        return instance;  
    }  
  
    public void doSomething() {  
        // useful code  
    }  
}
```

Variable holding our instance of the Singleton

Private constructor – may or may not contain any code

Being synchronized prevents multiple threads from accessing this concurrently

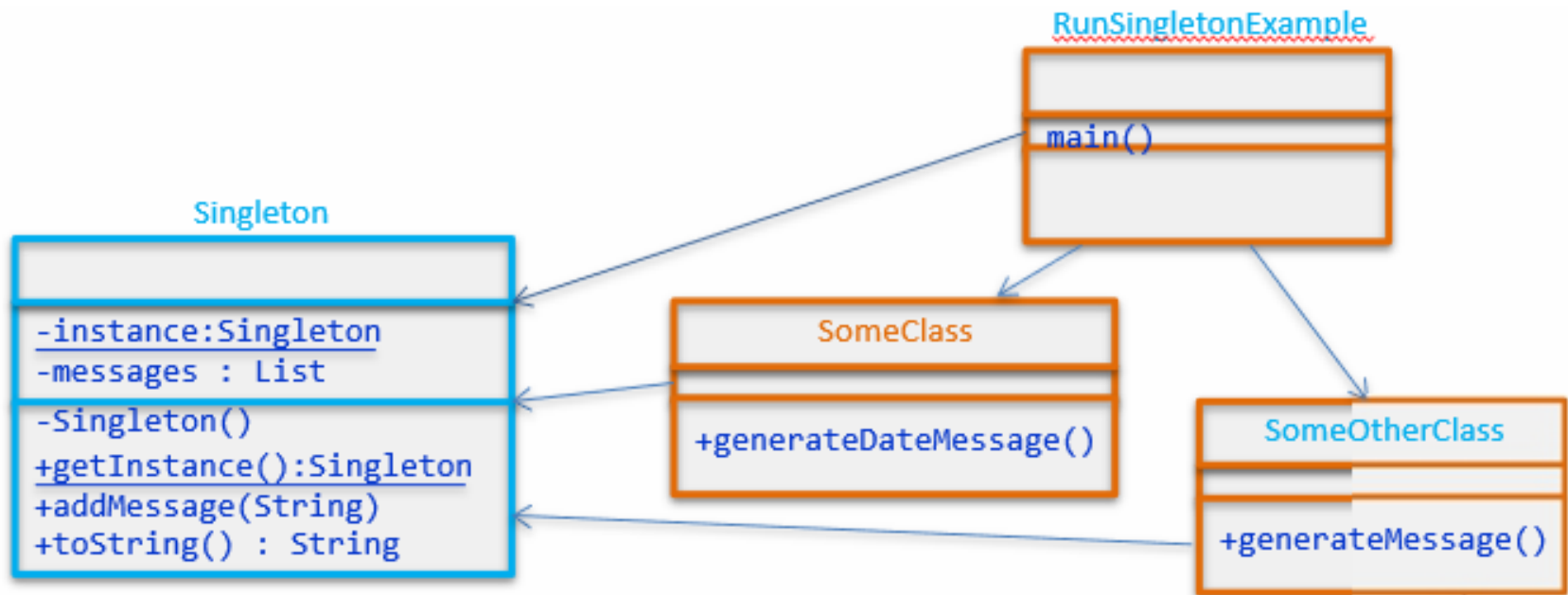
There will be one or more methods which actually do something

Singleton Example

- Let's create a program where a number of objects need to store messages in a List in memory
 - For example these might be text for SMS messages to be sent later
 - Only one instance of the List should exist and be shared by all the objects in the system. Access to it should be controlled

Singleton Example

- The UML class diagram:



Example – Singleton Class

```
package singletonpatternexample;
import java.util.*;
class Singleton {
    private List<String> messages;
    private static Singleton instance = null;
    private Singleton() {
        messages = new ArrayList<>();
    }
    public static synchronized Singleton getInstance() {
        if (instance == null)
            instance = new Singleton();
        return instance;
    }
    public synchronized void addMessage(String s) {
        messages.add(s);
    }
    public String toString() {
        return messages.toString();
    }
}
```

Example – Classes Using Singleton

```
package singletonpatternexample;
import java.util.Date;

public class SomeClass {

    public void generateDateMessage() {
        Singleton theOne = Singleton.getInstance();
        theOne.addMessage(new Date().toString());
    }
}
```

```
package singletonpatternexample;
public class SomeOtherClass {
    public void generateMessage() {
        Singleton theOne = Singleton.getInstance();
        theOne.addMessage("message 1");
        theOne.addMessage("message 2");
    }
}
```

Example – Main Application

```
package singletonpatternexample;

public class RunSingletonExample {

    public static void main(String[] args) {
        SomeClass sc = new SomeClass();
        SomeOtherClass soc = new SomeOtherClass();

        sc.generateDateMessage();
        soc.generateMessages();

        // Singleton sing = new Singleton(); // can't do that
        Singleton sing = Singleton.getInstance(); // do this instead

        System.out.println(sing.toString());
    }
}
```

Example output:

[Feb 10 08:37:36 GMT 2023, message 1, message 2]

Other Patterns



Template Design Pattern

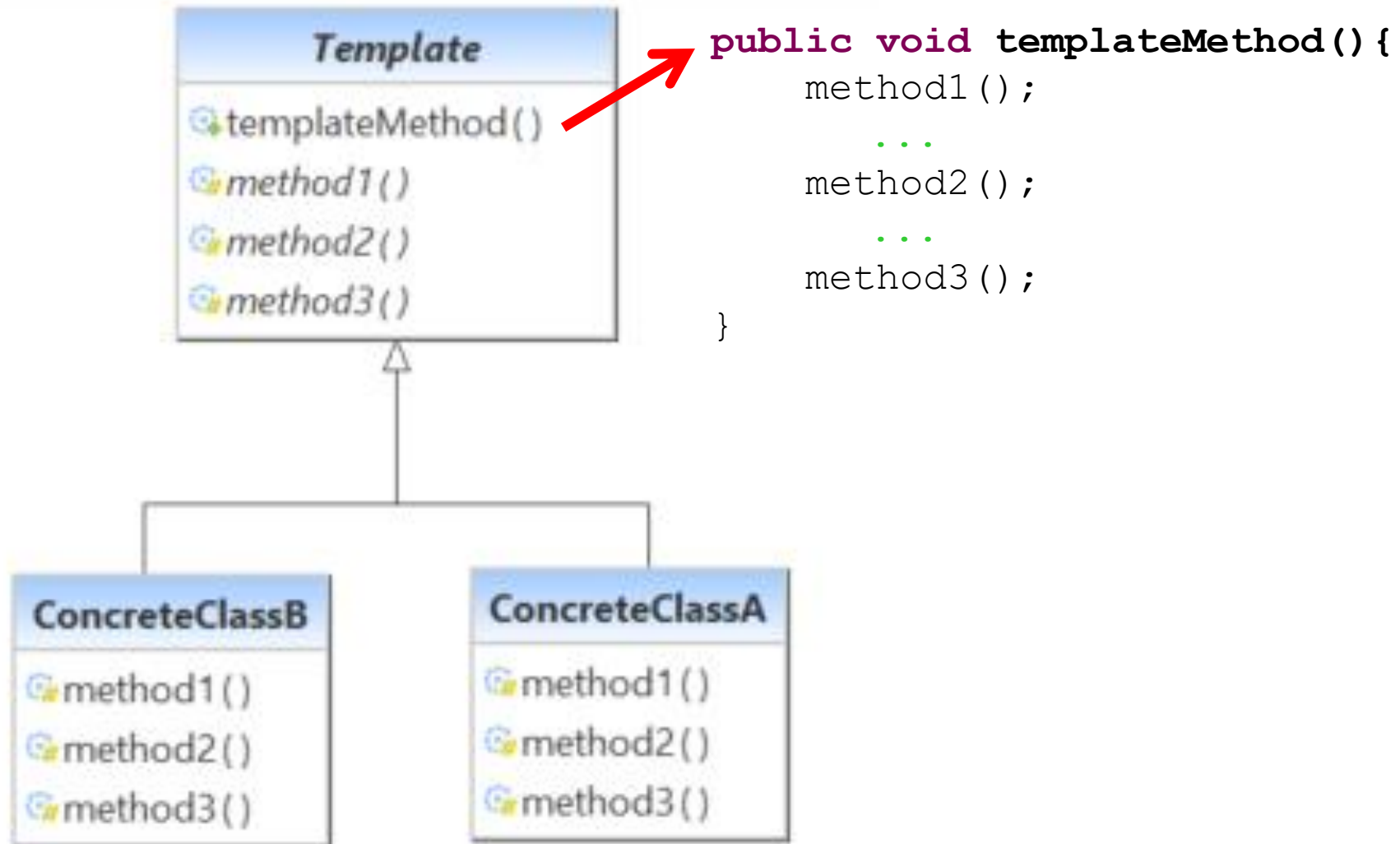
- A Behavioural Pattern
- Problem
 - You want to implement a reusable algorithm consisting of a number of steps, and the overall structure of algorithm is fixed, but the ways individual steps are carried out may vary
- Solution
 - Define the skeleton of an algorithm in a method (i.e. outline the steps that need to be carried out)
 - Defer some steps to subclasses.
 - Subclasses redefine certain steps of the algorithm without changing the algorithms structure

Template Design Pattern

○ Solution:

- Define an abstract class – the **Template class**
- The algorithm is defined in a concrete method - the **template method**
- The individual steps are abstract methods called by the template method
- Variations of the algorithm are achieved by creating subclasses of the template class that implement the abstract methods

Template – UML Diagram

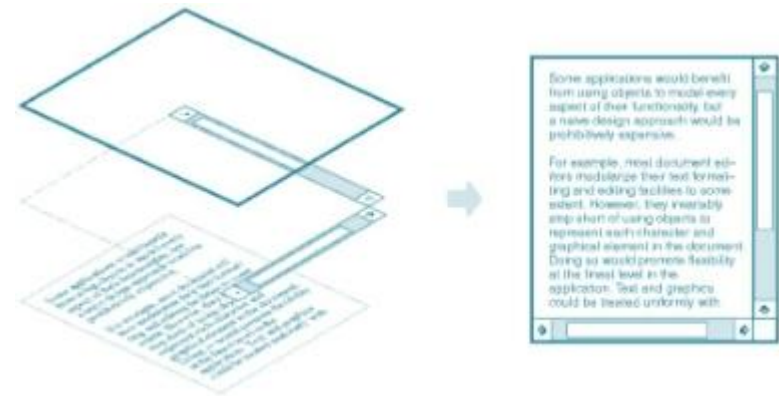


Decorator Pattern

- A Structural Pattern

- Problem:

- given a base object
- how to add functionality to an object
- making it easy to combine functionality
- avoiding a proliferation of subclasses



- Sometimes known as the Wrapper Pattern

- Objects that wrap around other objects to add useful features

Decorator Pattern

● Solution:

- an object that modifies behaviour of, or adds features to, another object
- decorator must maintain the common interface of the object it wraps up
- used so that we can add features to an existing simple object without needing to disrupt the interface that client code expects when using the simple object

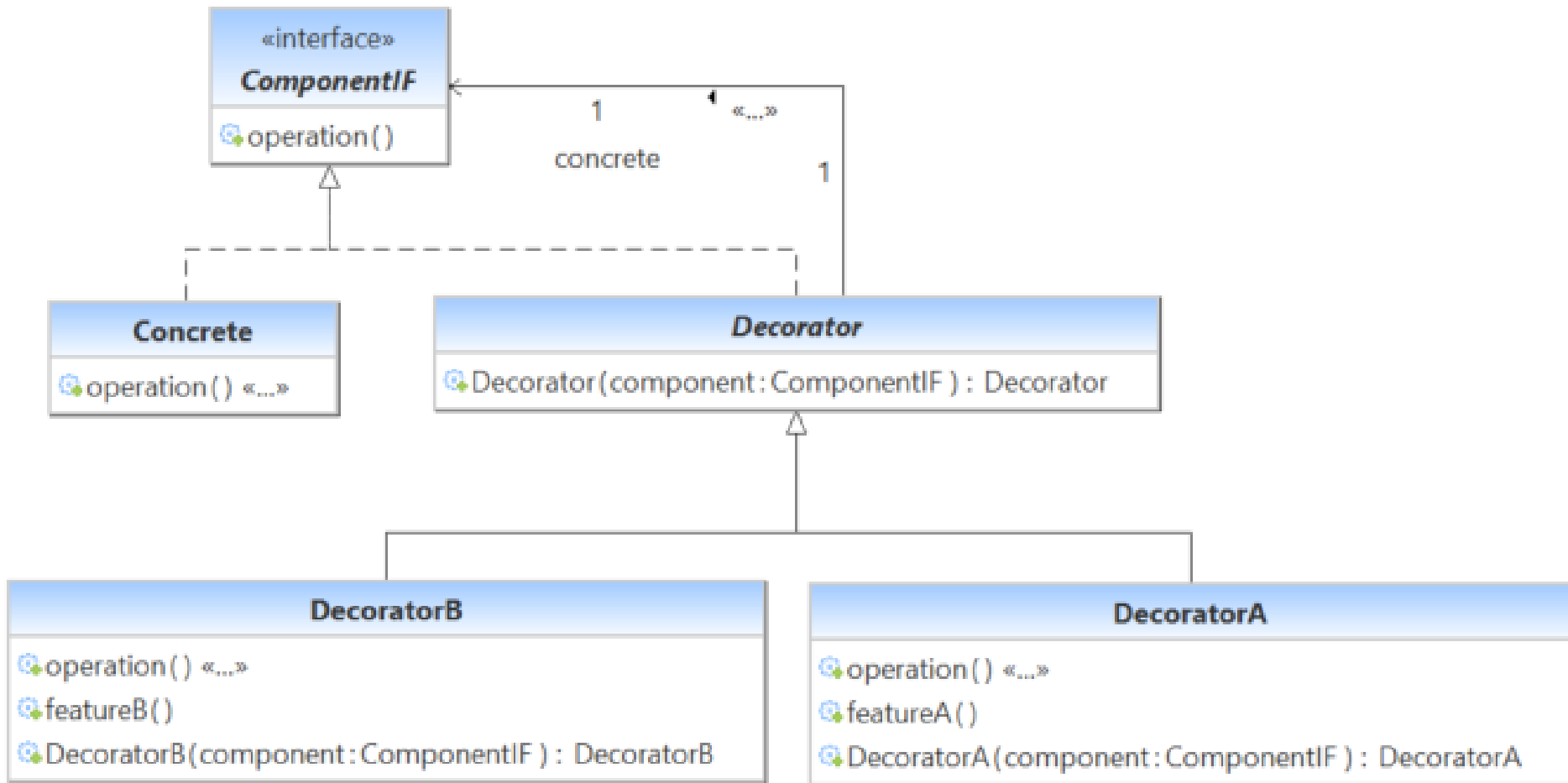
● Examples in Java:

- Adding designs, scroll bars and borders to GUI controls

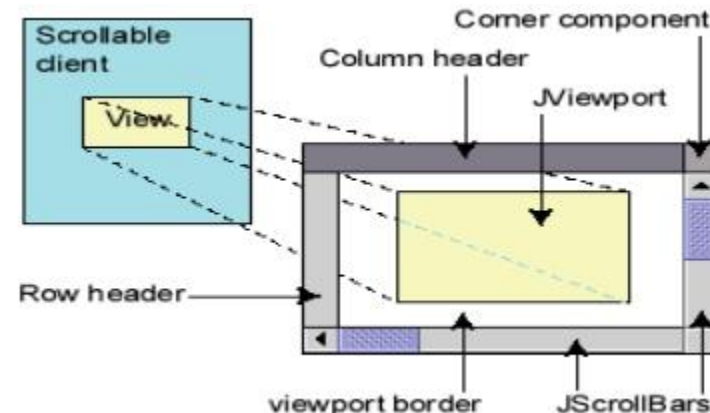
Decorator Pattern

- Have one or more Concrete classes that carry out the core operations
 - E.g. write to a file
- Have multiple Decorator classes that add features to the core operations
 - E.g. know how to compress data before writing to a file
- Make the Concrete classes and the Decorator classes implement the same interface
- Have the Decorator classes able to reference objects of the interface type

Decorator Pattern - UML



- Normal GUI components don't have scrollbars
 - JScrollPane is a container with scrollbars to which you can add any component to make it scrollable



```
// JScrollPane decorates GUI components
JTextArea area = new JTextArea(20, 30);
JScrollPane scrollPane = new JScrollPane(area);
contentPane.add(scrollPane);
```

Many More

- There are many more design patterns, we didn't have time to cover

End of week 8!